iOS 26.5 MTE Tagging Paths: Caller–Trigger Correlation and TOCTOU Preconditions

Kaya Ercihan

11.06.2026

Abstract

This document extends the initial iOS 26.0.1 static analysis by correlating the iOS 26.5 caller graph around `irg_mte_function` with the syscall-level trigger program `trigger_mte_paths.c`. The target helper is reverse-mapped to `vm_memtag_generate_and_store_tag()` at `0xFFFFFE000A8578A4`. Two userland-controlled pressure surfaces are identified: Mach IPC / zone allocation and BSD socket / Skywalk slab allocation. The security question is not whether `IRG`, `STG`, `ST2G`, or `STGM` are incorrect by themselves, but whether the helper's tag-store sequence is ordered with respect to subsequent object publication or recycling by an explicit barrier, a lock-based critical section, or a complete acquire/release protocol. Several traced zone paths contain no such identified end-to-end ordering. Skywalk remains a candidate trigger surface, but the surrounding slab-lock coverage must be resolved before treating its allocation and free paths as an overlapping MTE race surface. Runtime verification must therefore confirm the relevant tagging gates and establish actual kernel-level overlap between tagging, publication/consumption, and retagging.

1 Scope and notation

The concrete addresses in this report use the iOS 26.5 graph unless explicitly stated otherwise [1, 2, 3, 4].

Address convention. A row of the form `0xFFFF...: BL irg_mte_function` means that the enclosing recovered function directly calls the MTE helper at that program counter. The caller graph labels are recovered binary labels. The source names and line numbers are reverse-mapping annotations from the uploaded context.

Tag convention. Userland does not choose the final kernel allocation tag. The caller prepares `X0=address`, `X1=size`, and `X2=exclude-mask`; `irg_mte_function` then executes `IRG X0,X0,X2` to choose a random logical tag and stores the allocation tags over the covered granules with `STG`/`ST2G`/`STGM`. Therefore the required "tag to set" at the trigger level is the allocator/region policy bit: `zsflags->z_tag` for zones, or `SKR_MODE_MEMTAG` for Skywalk.

Interpretive convention. This report covers the caller-correlation and initial trigger-design stage of the investigation. A path described as a *candidate* denotes static reachability and a plausible concurrency surface, not a dynamically demonstrated race. Runtime validation must separately establish the relevant tagging gates, surrounding lock or acquire/release coverage, concurrent execution, and same-object overlap or reuse across the tag-store–publication boundary.

2 Seed function and common call shape

The graph centers on `irg_mte_function` at `0xFFFFFE000A8578A4`. The context maps it to `vm_memtag_generate_and_store_tag()` in `osfmk/vm/vm_memtag.h:72`; its job is to generate a random 4-bit tag, embed it in the pointer top byte, and store the tag to the tag granules [1, 5].

```
TST     X_addr, #0x100000000000000  ; select mask based on top-byte state
MOV     W8, #0xAAAB                 ; exclude mask A
MOV     W9, #0xD555                 ; exclude mask B
CSEL    X8, X9, X8, EQ
GMI     X2, X_addr, X8              ; X2 = exclude mask for IRG
MOV     X0, X_addr                  ; X0 = base address
MOV     X1, X_size                  ; X1 = allocation/object size
BL      irg_mte_function            ; X0 returns the tagged pointer
```

Listing 1: Common immediate pre-call pattern recovered from callers

```
FFFFFE000A8578A4: BTI   c
FFFFFE000A8578A8: IRG   X0, X0, X2
FFFFFE000A8578AC: AND   X9, X0, #0xFFFFFFFFFFFFFFF0
FFFFFE000A8578B0: ADD   X8, X1, X0
```

```
FFFFFE000A8578B4: ADD   X10, X8, #0xF
FFFFFE000A8578B8: AND   X8, X10, #0xFFFFFFFFFFFFFFF0
FFFFFE000A8578BC: SUB   X11, X10, X9
FFFFFE000A8578C0: CMP   X11, #0x3F
...
FFFFFE000A8578D4: STG   X9, [X9],#0x10
FFFFFE000A8578FC: STGM  X11, [X9]
FFFFFE000A857934: ST2G  X9, [X9],#0x20
FFFFFE000A857978: ST2G  X9, [X9],#0x20
FFFFFE000A857988: ST2G  X9, [X9],#0x20
FFFFFE000A857974: RET
```

Listing 2: iOS 26.5 `irg_mte_function` core: tag generation and tag stores, no visible DMB/DSB/ISB in the helper body

The helper itself contains the architectural MTE operations but not a general ordering fence. This matches the older iPhone 17 study, which found that the helper and its downstream worker do not contain explicit `DMB/DSB/ISB`; any required publication ordering must therefore be supplied by the caller, a higher-level lock discipline, or a complete acquire/release publication protocol [4].

3 Correlation matrix

Table 1: iOS 26.5 binary call sites correlated to source intent and userland trigger surface

Binary caller / BL	Subsystem	Mapped source role	Trigger	Ordering/trigger relevance
`sub_FFFFFE000A72F4FC` / `0xA72F658`	Zone	`zcram_memtag_init()` batch loop, `zalloc.c:3820`	zone expansion from `mach_port_allocate()` or `mach_msg()` pressure	Batch tagging of newly crammed zone memory; no complete end-to-end ordering protocol identified on the traced helper-to-publication segment.
`sub_FFFFFE000A72C2F4` / `0xA72CB00`	Zone	`zcram_and_lock()` / `zalloc_import()`, `zalloc.c:3873/5756`	zone refill/import under allocation pressure	Atomic RMW operations are present, but the refill-time publication protocol is not resolved.
`sub_FFFFFE000A72D974` / `0xA72DA48`	Zone	zone expand/refill	allocation pressure after cache miss	Single-range tagging during expansion; no explicit fence identified on the traced segment.
`sub_FFFFFE000A73265C` / `0xA7326A8`	Zone	`zone_tag_free_element()`, `zalloc.c:3806`	`mach_port_destruct()` / free-side churn	Re-tags freed elements; candidate for stale-consumer/retagging testing, but same-element reuse has not been established.
`sub_FFFFFE000A732704` / `0xA732804`	Zone	`zfree` free-side path	port/message teardown	Free-side retag path with no explicit fence on the traced segment.
`sub_FFFFFE000A732E34` / `0xA733088`	Zone	cached-drop/free path	cached object release	Barrier/lock-bearing zone edge; lower-priority candidate for initial runtime stress testing.
`sub_FFFFFE000AC10CFC` / `0xAC10DEC`	Zone/mbuf return	`__zcache_mark_valid()`, `zalloc.c:5824`	allocator cache return	Validity marking / existing-tag interaction; end-to-end publication protocol unresolved.
`sub_FFFFFE000AC8794C` / `0xAC87E0C`	Skywalk	`skmem_slab_alloc_locked()`, `skmem_slab.c:199`	`sendmsg()` / socket I/O	Allocation-side slab tagging in `skmem_slab_alloc_locked()`; overlap relevance depends on surrounding slab-lock coverage.
`sub_FFFFFE000AC8DDFC` / `0xAC8E3D0`	Skywalk	`skmem_slab_free_locked()`, `skmem_slab.c:593`	`recv()` completion / `close()`	Free-side slab retagging; initial candidate pending same-cache lock-serialization analysis.
`sub_FFFFFE000AC8EEF4` / `0xAC8F050`	Skywalk	pseudo/batch slab alloc, `skmem_slab.c:442/553`	connection batch pressure	Batch allocation reachability path; surrounding lock coverage across the relevant publication interval remains unresolved.

4 Zone allocator path: syscall to `irg_mte_function`

4.1 Path statement

For the zone path, the trigger program exercises two allocation surfaces and one teardown surface. First, it allocates many receive rights with `mach_port_allocate()`. Second, it sends Mach messages to those ports with `mach_msg()`, which reaches Mach trap #31 and allocates IPC message bodies through kalloc/zone paths. Finally, it destroys the ports with `mach_port_destruct()`, driving teardown through `zfree()` and free-list insertion. If the selected zone is tag-enabled, this path can reach free-side retagging.

```c
#define NUM_PORTS     512
#define MSG_BODY_SIZE 256

for (int i = 0; i < NUM_PORTS; i++) {
    mach_port_allocate(task, MACH_PORT_RIGHT_RECEIVE, &ports[i]);
```

```
    mach_port_insert_right(task, ports[i], ports[i], MACH_MSG_TYPE_MAKE_SEND);
}

for (int i = 0; i < NUM_PORTS; i++) {
    trigger_msg_t msg = {0};
    msg.header.msgh_bits = MACH_MSGH_BITS_SET(MACH_MSG_TYPE_COPY_SEND,0,0,0);
    msg.header.msgh_size = sizeof(msg);
    msg.header.msgh_remote_port = ports[i];
    mach_msg(&msg.header, MACH_SEND_MSG | MACH_SEND_TIMEOUT,
             sizeof(msg), 0, MACH_PORT_NULL, 100, MACH_PORT_NULL);
}

for (int i = 0; i < NUM_PORTS; i++) {
    mach_port_destruct(task, ports[i], -1, 0);
}
```

Listing 3: Relevant zone-trigger logic from `trigger_mte_paths.c`

4.2 Full correlation chain

Table 2: Zone path: from user call to MTE helper

Userland step	Kernel entry / source	Allocator transition	MTE edge
`mach_port_allocate()`	`mach_port_allocate()` in `osfmk/ipc/mach_port.c:514`	`ipc_port_alloc()` → `zalloc_id(ZONE_ID_IPC_PORT)` → `zalloc_ext()`	Cache miss / zone expansion reaches `zalloc_import()` / `zcram_and_lock()` then `zcram_memtag_init()` → `0xA72F658` / `irg_mte_function`.
`mach_msg()` send	Mach trap #31 `mach_msg_trap()` `osfmk/kern/syscall_sw.c:137`	`ipc_kmsg_alloc()` → `kalloc_data()` or `kalloc_type()` → `zalloc_ext()`, `zalloc.c:6157`	Zone expansion, refill, or page cram reaches `0xA72CB00`, `0xA72DA48`, or `0xA72F658`; returned objects are tagged before publication.
`mach_port_destruct()`	Port-right teardown / `ipc_port_dealloc()`	`zfree()` / free-list insertion	Free-side re-tagging reaches `zone_tag_free_element()` → `0xA7326A8` / `irg_mte_function`.

The zone guard is the per-zone security flag. If `z_tag` is not set for the selected zone, the call path can allocate/free objects without reaching MTE tagging. The source-level condition is summarized as:

```
// osfmk/kern/zalloc.c:3824
zone_security_flags_t *zsflags = &zone_security_array[zone_index(zone)];
if (!zsflags->z_tag) return;
```

Listing 4: Zone runtime gate

4.3 Mach IPC / zone allocator dynamic test method

The Mach IPC / zone allocator route is selected as the primary dynamic pressure surface because userland can repeatedly drive the `ipc_port` allocation class through allocation, use, and teardown. An allocation worker creates receive rights with `mach_port_allocate()`; optional `mach_msg()` traffic keeps ports active and adds message-side allocation pressure. A separate teardown worker releases previously created receive rights, driving the `zfree()` path and, when `zsflags->z_tag` is enabled, `zone_tag_free_element()` and free-side retagging.

The workers are released together after an initial warm-up and operate on independent batches or a handoff queue so that allocation and teardown phases overlap without a shared userland lock serializing both sides. This method targets same-zone lifetime churn across the allocation-side and free-side MTE edges identified in Table 2. The observable userland result is phase concurrency. A positive TOCTOU result still requires a tag fault, an allocator-consistency symptom, or kernel-side tracing that links the same element through retagging and later use.

5 Skywalk slab path: syscall to `irg_mte_function`

5.1 Path statement

The Skywalk path is triggered through loopback sockets. The trigger program creates a listener, connects many client sockets, sends data with `sendmsg()`, receives on the server side, and then closes all sockets. The allocation-side target is `skmem_slab_alloc_locked()`, and the free-side target is `skmem_slab_free_locked()`.

```c
#define NUM_SOCKETS   64
#define SEND_BUF_SIZE 4096
#define RECV_BUF_SIZE 4096

listen_fd = socket(AF_INET, SOCK_STREAM, 0);
bind(listen_fd, (struct sockaddr *)&addr, sizeof(addr));
listen(listen_fd, NUM_SOCKETS);

for (int i = 0; i < NUM_SOCKETS; i++) {
    client_fds[i] = socket(AF_INET, SOCK_STREAM, 0);
    connect(client_fds[i], (struct sockaddr *)&addr, sizeof(addr));
    server_fds[i] = accept(listen_fd, NULL, NULL);

    struct iovec iov = { .iov_base = send_buf, .iov_len = SEND_BUF_SIZE };
    struct msghdr mhdr = { .msg_iov = &iov, .msg_iovlen = 1 };
    sendmsg(client_fds[i], &mhdr, 0);
    recv(server_fds[i], recv_buf, RECV_BUF_SIZE, 0);
}

for (int i = 0; i < NUM_SOCKETS; i++) {
    close(server_fds[i]);
    close(client_fds[i]);
}
```

Listing 5: Relevant Skywalk-trigger logic from `trigger_mte_paths.c`

5.2 Full correlation chain

Table 3: Skywalk path: from user call to MTE helper

Userland step	Kernel entry / source	Allocator transition	MTE edge
`sendmsg()`	BSD syscall `sendmsg()`	`sendit()` → `sosend()` → Skywalk flow → `skmem_cache_alloc()`	`skmem_slab_alloc_locked()` at `skmem_slab.c:199` calls `vm_memtag_generate_and_store_tag()` on `bc->bc_addr` / `skm->skm_objsize`; binary edge `0xAC87E0C`.
`recv()`	receive path / socket buffer handling	RX-side packet buffer use and release	Can replenish or consume Skywalk buffers, increasing allocator/free overlap; related batch path `0xAC8F050`.
`close()`	file descriptor teardown	`skmem_cache_free()`	`skmem_slab_free_locked()` at `skmem_slab.c:593` re-tags freed buffers; binary edge `0xAC8E3D0`.

The Skywalk guard is the region mode bit. The path is only an MTE path when kernel tagging is compiled in and the Skywalk region has `SKR_MODE_MEMTAG` set:

```c
// bsd/skywalk/mem/skmem_slab.c:342 (alloc) and :654 (free)
region = sl->sl_cache->skm_region;
if (region->skr_mode & SKR_MODE_MEMTAG) {
    tagged_address = vm_memtag_generate_and_store_tag(bc->bc_addr,
                                                      skm->skm_objsize);
}
```

Listing 6: Skywalk runtime gate

The Skywalk route is retained as a second, independent userland-to-MTE reachability path. Its mapped allocation and free targets are `skmem_slab_alloc_locked()` and `skmem_slab_free_locked()`, so the surrounding slab-lock coverage must be assessed separately before interpreting concurrent socket activity as overlapping MTE operations. The primary dynamic race method in this report is therefore the Mach IPC / zone allocator design described in Section 4.3; Skywalk remains useful for reachability, allocator pressure, and fault/noise comparison.

6 Parallel triggering model

`trigger_mte_paths.c` currently executes the correlated trigger paths sequentially. For runtime verification of the TOCTOU hypothesis, the primary change is to parallelize the Mach IPC / zone allocator route using separate allocation and teardown workers. The workers must overlap in time without a shared userland lock serializing both lifetime transitions. Skywalk may execute alongside this test as an independent reachability and allocator-pressure path, but it is not required for the primary zone-race method.

```
shared atomic_bool start = false;
shared concurrent_queue ports;

zone_alloc_worker:
  wait(start);
  repeat {
    p = allocate_receive_right();
```

```
    optionally_send_mach_message(p);
    enqueue(ports, p);
  }

zone_teardown_worker:
  wait(start);
  repeat {
    p = try_dequeue(ports);
    if (p != none)
      release_receive_right(p);
  }

skywalk_control_worker:
  wait(start);
  repeat { connect_send_recv_close(); }

main:
  warm_up_ipc_port_zone();
  create N zone worker pairs and optional Skywalk controls;
  start = true;
  run a bounded pressure window;
  collect counts, faults, and allocator diagnostics;
```

Listing 7: Initial Mach IPC / zone allocator parallel-pressure model → intentionally pseudocode-level

The practical target is not "many syscalls" by itself, but repeated lifetime transitions within the same zone allocation class:

1. **Allocation phase.** Create Mach receive rights and, where required, add message traffic so that allocation pressure reaches zone refill or expansion and the allocation-side MTE edges.
2. **Live/use phase.** Keep allocated ports or queued messages live long enough to exercise the corresponding publication and consumer paths.
3. **Teardown/retag phase.** Release previously allocated receive rights concurrently with continued allocation. When the target zone is tag-enabled, teardown can reach `zone_tag_free_element()` and assign a new tag during the object lifetime transition.

The run should use independent handoff queues rather than one shared lock around all operations. One worker group continuously allocates receive rights and optionally sends Mach messages, while another group tears down previously allocated rights. Optional Skywalk control workers may execute a connect/send/recv/close loop in parallel. The directly measurable userland result is worker-phase overlap. The security-relevant interval is narrower: the period from tag-store issuance to later same-object tag-dependent publication or consumption. A valid positive result therefore requires an observable tag fault or allocator-consistency symptom, or kernel-side tracing that links the same element through retagging and later use. Worker-phase overlap by itself is pressure evidence rather than proof of the hypothesized race.

7 Preconditions checklist

Table 4: Runtime conditions required before interpreting the stress result

Condition	Meaning
`ZSECURITY_CONFIG (ZONE_TAGGING)`	Zone tagging must be compiled in. Without it, the zone code can skip the MTE path entirely.
`zsflags->z_tag==1`	The concrete zone must be tag-enabled. For the trigger program this should hold for the IPC/kalloc zones being pressured.
`CONFIG_KERNEL_TAGGING`	Skywalk memory tagging code must be compiled in.
`region->skr_mode & SKR_MODE_MEMTAG`	The Skywalk region must request MTE tags for this route to exercise the tagging helper. Otherwise, `sendmsg()` provides allocator pressure without exercising the MTE path.
`dword_FFFFFE0007C8CCB0 != 0`	iOS 26.5 graph-side global follow-up count/control used by several post-helper loops; if zero, the bulk follow-up `vm_memtag_store_tag()` loop is skipped.
No spanning external synchronization	The relevant tag-store–publication boundary must not be covered by a higher-level lock or a complete acquire/release protocol outside the recovered path.
Parallel overlap	Allocation, use/consumption, and teardown phases must execute concurrently on separate worker threads; execution on different CPU cores is desirable for stressing inter-core ordering but is not assumed.

8 Conclusion

The iOS 26.5 graph identifies two userland-controlled pressure routes into the same MTE tag-generation helper: Mach IPC through the zone allocator and BSD socket I/O through Skywalk slabs. In both routes the logical tag is generated by `IRG` inside `irg_mte_function` and written with `STG`/`ST2G`/`STGM`; userland controls allocator pressure and object lifetime transitions rather than the kernel tag value.

The Mach IPC / zone allocator route is used as the primary dynamic test surface because separate allocation and teardown workers can repeatedly drive same-zone lifetime transitions. Skywalk is retained as an independent reachability and allocator-pressure path, but is not required for the primary zone-race method. A meaningful positive TOCTOU result requires an observable misordering symptom, such as a tag fault, stale tagged-pointer use, or allocator-consistency failure, or kernel-side tracing that links the same element through retagging and subsequent use. Worker-phase overlap alone demonstrates concurrency pressure rather than the hypothesized same-object race, and the relevant tagging gates must be confirmed enabled before the runtime result is interpreted.

References

[1] Kaya Ercihan. *Caller Graph Analysis: irg_mte_function, iOS 26.5 Kernelcache*. IDA Pro disassembly and custom TOCTOU barrier analysis. 2026.

[2] Kaya Ercihan. *caller_graph_mie_ios26.5.pdf*. Caller graph for iOS 26.5 kernelcache; seed function at 0xFFFFFE000A8578A4. 2026.

[3] Kaya Ercihan. *trigger_mte_paths.c*. Userland trigger source for zone allocator and Skywalk slab MTE paths. 2026.

[4] Kaya Ercihan. *Investigating a Potential TOCTOU in the iOS 26.0.1 MTE Tagging Path*. Static reverse-engineering assessment of MTE caller ordering. 2026.

[5] Arm Limited. *Arm Architecture Reference Manual for A-profile architecture*. Version Issue M.a. Document DDI 0487; architectural reference for MTE instructions and memory-ordering barriers. 2026. URL: `https://developer.arm.com/documentation/ddi0487/maa` (visited on 06/11/2026).